Homework Assignment 4: Nonlinearly Constrained Optimization

- **Learning outcome:** By completing this assignment, you will be able to check the second-order sufficient conditions on the tangent subspace by hand, and implement and compare penalty, augmented Lagrangian, SQP, and GRG algorithms in code.
- You may use either MATLAB or Python for the parts that require coding. The course notebooks needed for Questions 2, 3, and 4 are linked in those questions, and a template for Question 4 is provided in both languages: `HW4_GRG_template.ipynb` (Python) and `HW4_GRG_template.m` (MATLAB).
- Your submission should include: (i) a PDF report named `<McGill_ID>_HW4.pdf` explaining your solution approach, and (ii) either a published MATLAB script or a Python notebook.
- For Questions 2, 3, and 4 you may work in teams, but you must declare whom you worked with and submit your own work (no group submissions).
- Throughout, use the negative null form of the lectures, i.e., $\min f(\mathbf{x})$ subject to $\mathbf{h}(\mathbf{x}) = \mathbf{0}$ and $\mathbf{g}(\mathbf{x}) \leq \mathbf{0}$, with Lagrangian $L = f + \boldsymbol{\lambda}^{\mathrm{T}}\mathbf{h} + \boldsymbol{\mu}^{\mathrm{T}}\mathbf{g}$.

---

1. **Second-order conditions on the tangent subspace** (Exercise 5.13, Papalambros & Wilde, 3rd edition)

   This question is to be done by hand. Consider the problem

   $$\begin{aligned} \min_{x_1,x_2} \quad & f = \frac{1}{2}\left[(x_1-1)^2 + x_2^2\right] \\ \text{subject to} \quad & h = -x_1 + b\,x_2^2 = 0, \end{aligned} \tag{1}$$

   where b is a fixed scalar.

   (a) Show that $\mathbf{x} = (0,0)^{\mathrm{T}}$ satisfies the first-order necessary conditions of Problem (1) for every value of b, and find the corresponding Lagrange multiplier.

   (b) Using the second-order conditions on the subspace tangent to h, find the values of b for which $\mathbf{x} = (0,0)^{\mathrm{T}}$ is a local minimizer. Treat the borderline value of b separately.

   (c) Explain your result graphically for $b = 1/4$ and $b = 1$ by sketching the contours of f and the constraint curve. For $b = 1$, find the actual minimizers.

2. **Penalty and augmented Lagrangian methods** (Example 5.9, Papalambros & Wilde, 3rd edition)

   Consider the problem

   $$\begin{aligned} \min_{x_1,x_2} \quad & f(x_1,x_2) = x_1^2 + (x_2-3)^2 \\ \text{subject to} \quad & g(x_1,x_2) = x_2^2 - 2x_1 \leq 0. \end{aligned} \tag{2}$$

   Become familiar with the following course notebooks:

- L09_penalty_and_barrier_methods.ipynb:
  `run_penalty` minimizes $T(\mathbf{x}, r) = f + \frac{1}{r}\big(\max\{0, g\}\big)^2$ for a decreasing sequence $r_k = r_0\beta^k$.
- L09_augmented_lagrangian_method.ipynb:
  `run_augmented_lagrangian` minimizes $T(\mathbf{x}, r, \mu) = f + \mu g + \frac{1}{r}\big(\max\{0, g\}\big)^2$ and updates $\mu_{k+1} = \mu_k + \frac{2}{r}g(\mathbf{x}_k)$.

Replace the example problem in the notebooks with Problem (2). MATLAB users may reproduce the same steps in MATLAB (e.g., with `fminunc` for the unconstrained subproblems).

(a) By hand, use the KKT conditions to show that the minimizer satisfies $x_2^3 + 2x_2 - 6 = 0$ with $x_1 = x_2^2/2$ and $\mu = x_1$. Solve this cubic numerically and report $\mathbf{x}^*$, $f(\mathbf{x}^*)$, and μ^*. Explain why $\mathbf{x}^*$ is the global minimizer.

(b) **Penalty method.** Starting from $\mathbf{x}_0 = (0, 3)^{\mathrm{T}}$, run the penalty method with $r_k = 0.1^k$, $k = 0, 1, \ldots, 6$. Show that $\mu_k = \frac{2}{r_k}\max\{0, g(\mathbf{x}_k)\}$ is an estimate of the Lagrange multiplier. For each k, report $\mathbf{x}_k$, $f(\mathbf{x}_k)$, $g(\mathbf{x}_k)$, μ_k, $\|\mathbf{x}_k - \mathbf{x}^*\|$, and the condition number of $\nabla^2_{xx}T(\mathbf{x}_k, r_k)$ (analytically, or with `numerical_hessian` from L06_newton_and_quasi_newton.ipynb). Are the iterates feasible? What happens to the accuracy of the unconstrained solutions as $r_k \to 0$, and why?

(c) **Augmented Lagrangian method.** Starting from $\mathbf{x}_0 = (0, 3)^{\mathrm{T}}$ with $\mu_0 = 0$ and a *fixed* $r = 1$, run eight outer iterations of the augmented Lagrangian method. Report μ_k, $\mathbf{x}_k$, $g(\mathbf{x}_k)$, and $\|\mathbf{x}_k - \mathbf{x}^*\|$. Compare with the minimizer of the plain penalty function $T(\mathbf{x}, r)$ at the same fixed $r = 1$, and explain why repeating the penalty minimization with r fixed cannot improve on it.

(d) Plot the sequences of unconstrained minimizers of both methods on a contour plot of f that also shows the constraint boundary, and plot $\|\mathbf{x}_k - \mathbf{x}^*\|$ versus k for all runs on a semi-log scale (e.g., `semilogy` in MATLAB or `plt.semilogy` in Python). Compare the methods.

3. **Sequential quadratic programming** (Exercise 6.32, Papalambros & Wilde, 3rd edition)

Consider the Rosen–Suzuki problem (Hock and Schittkowski, Problem 43)

$$\begin{aligned} \min_{\mathbf{x}\in\mathbb{R}^4} \quad & f = x_1^2 + x_2^2 + 2x_3^2 + x_4^2 - 5x_1 - 5x_2 - 21x_3 + 7x_4 \\ \text{subject to} \quad & g_1 = x_1^2 + x_2^2 + x_3^2 + x_4^2 + x_1 - x_2 + x_3 - x_4 - 8 \le 0, \\ & g_2 = x_1^2 + 2x_2^2 + x_3^2 + 2x_4^2 - x_1 - x_4 - 10 \le 0, \\ & g_3 = 2x_1^2 + x_2^2 + x_3^2 + 2x_1 - x_2 - x_4 - 5 \le 0. \end{aligned} \tag{3}$$

(a) By hand, show that $\mathbf{x}^* = (0, 1, 2, -1)^{\mathrm{T}}$ is a KKT point: identify the active constraints, compute the Lagrange multipliers $\boldsymbol{\mu}^*$, and report $f(\mathbf{x}^*)$. Explain why $\mathbf{x}^*$ is the global minimizer.

(b) Adapt `sqp` from Part 2 of L09_sqp.ipynb to Problem (3). Note that the notebook's `sqp` is written for a 2-variable, 4-constraint problem, and that its matrix `Wk` is the Hessian of

the Lagrangian *of that problem*. You must replace it with

$$\mathbf{W}_k = \nabla^2 f + \sum_{j=1}^{3} \mu_{j,k} \nabla^2 g_j,$$

where $\mu_{j,k}$ are the QP multipliers from the previous iteration ($\mu_{j,0} = 0$). Starting from $\mathbf{x}_0 = \mathbf{0}$, terminate when $\|\partial \mathbf{x}_k\| \le 10^{-8}$, when the merit-function line search can no longer find a decrease, or after $k_{\max} = 200$ iterations. For each iteration, report $\mathbf{x}_k$, $f(\mathbf{x}_k)$, the maximum constraint violation $\max\{0, \max_j g_j(\mathbf{x}_k)\}$, $\|\nabla_x L(\mathbf{x}_k, \boldsymbol{\mu}_k)\|$ (with $\boldsymbol{\mu}_k$ the multipliers used to form $\mathbf{W}_k$), and the step size accepted by the merit-function line search. Report the final $\mathbf{x}$, $\boldsymbol{\mu}$, and active set, and compare with part (a).

(c) Solve the same problem from the same initial guess with `scipy.optimize.minimize` (method `SLSQP`) or `fmincon` (MATLAB, algorithm `sqp`). Compare the solution and the number of iterations.

4. **Generalized reduced gradient (GRG) algorithm** (Exercise 6.31, Papalambros & Wilde, 3rd edition)

Consider the problem

$$\begin{aligned} \min_{\mathbf{x}\in\mathbb{R}^5} \quad & f = (x_1 - x_2)^2 + (x_2 - x_3)^2 + (x_3 - x_4)^2 + (x_4 - x_5)^2 \\ \text{subject to} \quad & h_1 = x_1 + x_2^2 + x_3^3 - 3 = 0, \\ & h_2 = x_2 - x_3^2 + x_4 - 1 = 0, \\ & h_3 = x_1 x_5 - 1 = 0, \end{aligned} \tag{4}$$

with initial guess $\mathbf{x}_0 = \big(2,\ \sqrt{2},\ -1,\ 2 - \sqrt{2},\ 0.5\big)^{\mathrm{T}}$. Use $\mathbf{x}_d = (x_2, x_3)^{\mathrm{T}}$ as decision variables and $\mathbf{x}_s = (x_1, x_4, x_5)^{\mathrm{T}}$ as state variables.

(a) By hand, verify that $\mathbf{x}_0$ is feasible. Write $\nabla_{x_s}\mathbf{h}$ and $\nabla_{x_d}\mathbf{h}$, show that $\det(\nabla_{x_s}\mathbf{h}) = x_1$, and state where in the design space this choice of state variables fails. Can this happen at a feasible point? What happens to $\nabla_{x_s}\mathbf{h}$ as a feasible point approaches that region? Compute the reduced gradient $\nabla w^{\mathrm{T}} = \nabla_{x_d} f^{\mathrm{T}} - \nabla_{x_s} f^{\mathrm{T}} (\nabla_{x_s}\mathbf{h})^{-1} \nabla_{x_d}\mathbf{h}$ at $\mathbf{x}_0$.

(b) The template `HW4_GRG_template` already provides Problem (4), the reduced gradient (`reduced_gradient`), and the Newton–Raphson restoration of the states (`restore_state`). Use `grg` from Section 3 of `L09_generalized_reduced_gradient.ipynb` as a guide (it handles one decision and one state variable) and complete the function `grg` in the template for vector-valued $\mathbf{x}_d$ and $\mathbf{x}_s$. Each iteration must:

(1) take the step $\partial \mathbf{x}_d = -\alpha_k \nabla w_k$ in the decision space;

(2) predict the states with the linear update $\hat{\mathbf{x}}_{s,k+1} = \mathbf{x}_{s,k} - (\nabla_{x_s}\mathbf{h})_k^{-1} (\nabla_{x_d}\mathbf{h})_k \, \partial \mathbf{x}_d$;

(3) restore feasibility with `restore_state`, starting from $\hat{\mathbf{x}}_{s,k+1}$;

(4) choose α_k by Armijo backtracking on f: start from $\alpha = 1$ and halve it ($\beta = 0.5$) until the restored point satisfies $f(\mathbf{x}_{k+1}) \le f(\mathbf{x}_k) - \varepsilon\, \alpha \, \|\nabla w_k\|^2$ with $\varepsilon = 10^{-4}$, rejecting any trial step for which the restoration fails.

Terminate when $\|\nabla w\| \le 10^{-6}$ or after $k_{\max} = 5000$ iterations. Report the final $\mathbf{x}$, $f(\mathbf{x})$, $\|\nabla w\|$, $\|\mathbf{h}\|$, and the number of iterations.

(c) In this problem the states can be eliminated explicitly ($x_1 = 3 - x_2^2 - x_3^3$, $x_4 = 1 - x_2 + x_3^2$, $x_5 = 1/x_1$). Use this to plot the contours of the reduced objective $w(x_2, x_3)$ and show all GRG iterates (including the initial guess) on it. Also plot $\|\nabla w\|$ versus k on a semi-log scale (e.g., `semilogy` in MATLAB or `plt.semilogy` in Python).

(d) Compute the Lagrange multipliers at your final point from $\boldsymbol{\lambda}^{\mathrm{T}} = -\nabla_{x_s} f^{\mathrm{T}} (\nabla_{x_s} \mathbf{h})^{-1}$ and interpret their values. Compare your solution with that of `scipy.optimize.minimize` (method `SLSQP`) or `fmincon` (MATLAB, algorithm `sqp`) from the same initial guess.

Getting help on functions

- **MATLAB:** type `help fmincon` (or `doc fmincon`) for a description of the function and its inputs and outputs.
- **Python:** run `help(scipy.optimize.minimize)` after `import scipy.optimize` for the same information.